Online Training Program
Online Training Program
Online Training
Online Training

ASP.NET WEB API - Basics
Creating ASP.NET Web API Project
Creating ASP.NET Web API Project - Continued
How to add Controller in Web API
How to add Folder in Web API
How to use HttpRequestMessage in Web API
ASP.NET Web API using Entity Framework
Data Representation in Web API
Media Type Formatter in Web API
How to Implement GET Method in Web API
How to Implement POST Method in Web API
How to Implement PUT Method in Web API
How to Implement DELETE Method in Web API
Parameter Binding in Web API
Parameter Binding in Web API Source Code
Calling Web API Service in a Console Application
Cross Origin Resource Sharing (CORS)

WEB API Routing and HTTP Routing
Attribute Routing in ASP.NET Web API
Routing and Action Selection in Web API
ASP.NET Web API Message Handlers
Message Handlers in ASP.NET Web API
Basic Authentication Using Message Handler in Web API
HTTP Cross Message Handler in Web API
Route Parameters and Route Constraints in Web API
Route Prefix and Route Constraints in Web API
Attribute Routing Route Constraints
Route Parameter and Route Constraints in Attribute Routing

ASP.NET WEB API - Security
Basic Authentication in ASP.NET Web API
Basic Authentication in ASP.NET Web API
Basic Auth in ASP.NET Web API - Source Code
Role Based Basic Authentication in Web API
Role Based Basic Authentication in Web API using Custom Header
Basic Authentication Using Custom Header
Token Based Authentication in Web API
Token Based Authentication in Web API - Source Code
Token Based Authentication in ASP.NET Web API
Token Refresh Token in Web API
Consume Refresh Token in Web API
HMAC Authentication in Web API

Web API Versioning
Web API Versioning Using URI
Web API Versioning using Query String Parameter
Web API Versioning Using Custom Header
Web API Versioning Using Accept Header
Web API Versioning Using Custom Media Type

Advanced Concepts
Web API Tracing Using Tracelog
Dependency Injection
Change HTTP to Ajax API

Miscellaneous
ASP.NET Web API Interview Questions
Web API Books

Reading
Read Recommended ASP.NET Web API Books

Role-Based Basic Authentication in Web API

Back to ASP.NET Web API Tutorial for Beginners and Professionals

Role-Based Basic Authentication in Web API

In this article, I am going to discuss how to implement **Role Based Basic Authentication in ASP.NET Web API** Applications. Please read our last article before proceeding to this article, where we discussed How to implement ASP.NET Web API Basic Authentication with an example. As part of this article, we are going to discuss the following contents related to authentication and authorization.

- **Why do we need Role-Based Authentication?**
- **Implementing Role-Based Basic Authentication in Web API?**
- **Testing the Role-Based Basic Authentication using Postman.**
- **What are the advantages and disadvantages of using BASIC Authentication in Web API?**

Why do we need Role-Based Authentication?

Let us understand this with an example. As shown in the below image, we have three resources i.e. **GetAllMedicalEmployees**, **GetAllFamilyEmployees** and **GetAllDeathEmployees** in our Web API.

```
[Route("api/v2/MedicalEmployees")]
public IEnumerable<Employee> GetAllMedicalEmployees()
{
}

[Route("api/v2/FamilyEmployees")]
public IEnumerable<Employee> GetAllFamilyEmployees()
{
}

[Route("api/v2/DeathEmployees")]
public IEnumerable<Employee> GetAllDeathEmployees()
{
}
```

In our application, let say we have two types of **Roles** i.e. **Admin** and **Supervisor**and we can run business requirement.

- Only the admin user has the Role Admin can access only to the GetAllMedicalEmployees resource.
- The users who have the Role Supervisor can access only for GetAllFamilyEmployees resource.
- The GetAllDeathEmployees resource can be accessed by both the Admin and Supervisor resource.

In order to achieve this we need to implement Role Based Authentication in ASP.NET Web API.

Implementing Role-Based Basic Authentication in Web API.

In order to secure our entire Web API application with the name **RoleBasedBasicAuthenticationWEBAPI**. Then Add the following Model and Employee models to the Models folder.

Client.cs

```
namespace RoleBasedBasicAuthenticationWEBAPI.Models
{
    public class Client
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Secret { get; set; }
        public string Active { get; set; }
        public int? RefreshTokenLifeTime { get; set; }
        public string Allowedorigin { get; set; }
    }
}
```

Employee.cs

```
namespace RoleBasedBasicAuthenticationWEBAPI.Models
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int? Salary { get; set; }
        public string Address { get; set; }
    }
}
```

And we need to add the Identity and Owin Framework. Open the within the Double Click.

IdentityM.cs

```
namespace RoleBasedBasicAuthenticationWEBAPI.Models
{
    public class IdentityM
    {
        // You can add profile data for the user here. Dont like putting sensitive stuff in
        // but it's convenient to have stuff like roles available on Basic Authentication
        // Profile properties are serialized to JSON and stored in the database.
        public IdentityUser user = new IdentityUser();

        public IdentityM(string name)
        {
            var roles = new Dictionary<
            {
                {"Admin", "Admin"},
                {"Supervisor", "Supervisor"},
                {"Guest", "Guest"},
                {"Guest1", "Guest1"}
            };

            foreach (var r in roles)
            {
                if (name == r.Key)
                    user.Roles.Add(new IdentityUserRole
                    {
                        RoleId = r.Key,
                        UserId = name
                    });
            }

            user.UserName = name;
            user.PasswordHash = new PasswordHasher().HashPassword("Test@123");
            user.SecurityStamp = Guid.NewGuid().ToString("D");
            user.Id = Guid.NewGuid.ToString("D");
            user.Email = name + "@test.com";
            user.EmailConfirmed = true;
            user.PhoneNumber = "1234567890";
            user.PhoneNumberConfirmed = true;
            user.TwoFactorEnabled = false;
            user.LockoutEnabled = false;
            user.AccessFailedCount = 0;
        }
    }
}
```

Protected.cs

```
namespace RoleBasedBasicAuthenticationWEBAPI.Models
{
    public class Protected
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int? Salary { get; set; }
        public string Address { get; set; }
    }
}
```

And we need to add the Identity and Owin Framework. Open the within the Double Click.

UserDetails.cs

```
namespace RoleBasedBasicAuthenticationWEBAPI.Models
{
    public class UserDetails
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int? Salary { get; set; }
        public string Address { get; set; }
        public int? Age { get; set; }
        public int? Phone { get; set; }
        public int? Pin { get; set; }
        public int? Country { get; set; }
        public int? State { get; set; }
        public int? City { get; set; }
        public int? Gender { get; set; }
        public int? Publish { get; set; }
        public int? Active { get; set; }
        public int? Deleted { get; set; }
        public int? Approved { get; set; }
        public int? Rejected { get; set; }
        public int? Pending { get; set; }
        public int? Total { get; set; }
        public int? Page { get; set; }
        public int? Size { get; set; }
        public int? Sort { get; set; }
        public int? Order { get; set; }
        public int? Search { get; set; }
        public int? Type { get; set; }
        public int? Value { get; set; }
        public int? Text { get; set; }
        public int? Date { get; set; }
        public int? Time { get; set; }
        public int? Image { get; set; }
        public int? Video { get; set; }
        public int? Audio { get; set; }
        public int? Document { get; set; }
        public int? Other { get; set; }
        public int? Error { get; set; }
        public int? Success { get; set; }
        public int? Warning { get; set; }
        public int? Info { get; set; }
        public int? Question { get; set; }
        public int? Yes { get; set; }
        public int? No { get; set; }
        public int? Cancel { get; set; }
        public int? Ok { get; set; }
        public int? Save { get; set; }
        public int? Delete { get; set; }
        public int? Edit { get; set; }
        public int? View { get; set; }
        public int? Print { get; set; }
        public int? Download { get; set; }
        public int? Upload { get; set; }
        public int? Share { get; set; }
        public int? Copy { get; set; }
        public int? Paste { get; set; }
        public int? Cut { get; set; }
        public int? Undo { get; set; }
        public int? Redo { get; set; }
        public int? Refresh { get; set; }
        public int? Reset { get; set; }
        public int? Clear { get; set; }
        public int? Search { get; set; }
        public int? Filter { get; set; }
        public int? Sort { get; set; }
        public int? Order { get; set; }
        public int? Page { get; set; }
        public int? Size { get; set; }
        public int? Total
```